

A Study on Transfer Learning TinyML-Based Intrusion Detection Framework on IoT Devices

Jedidah Mwaura

*Graduate School of Computer Science
and Systems Engineering
Kyushu Institute of Technology
Fukuoka, Japan*

mwaura.mweru-jedidah901@mail.kyutech.jp

Shunsuke Araki

*Faculty of Computer Science
and Systems Engineering
Kyushu Institute of Technology
Fukuoka, Japan*

araki@csn.kyutech.ac.jp

Muhammad Bisri Musthafa

*Graduate School of Environmental,
Life, Natural Science and Technology
Okayama University
Okayama, Japan*

bisrimusthafa@s.okayama-u.ac.jp

Samsul Huda

*Green Innovation Center
Okayama University
Okayama, Japan*

shuda@okayama-u.ac.jp

Yasuyuki Nogami

*Graduate School of Environmental,
Life, Natural Science and Technology
Okayama University
Okayama, Japan*

yasuyuki.nogami@okayama-u.ac.jp

Abstract—Currently, there are many ongoing attempts to incorporate transfer learning into Intrusion Detection Systems. Transfer learning leverages on the knowledge of previous models thus reducing training time, improving accuracy and making it possible to train new models with limited data. However, it is difficult to incorporate such a technique into IoT devices with limited computational power. In this paper, we propose a Transfer Learning TinyML-based IDS framework with feature selection to be implemented on a microcontroller for detecting attacks on IoT devices. To implement the proposal, we leveraged knowledge achieved from our previous DDoS attack detection model to create a new transfer learning framework that could detect various IoT attacks such as malware attacks and zero-day attacks. To test the transfer learning framework, we trained it using two separate sets of datasets each containing different attacks. After each training, we evaluated the original model, converted it into a format compatible with a microcontroller and evaluated it further. The TL TinyML-based framework was evaluated based on its accuracy, model sizes and average inference times. As the results confirmed, the framework performed well and can successfully be implemented on a microcontroller without depleting its resources.

Index Terms—IoT devices, TinyML, IoT Malware, Transfer Learning, Cross Validation

I. INTRODUCTION

The rise, growth and affordability of the Microcontrollers capable of running computations on low powered devices have significantly boosted the growth of IoT devices. These Microcontrollers enable IoT devices to operate efficiently with minimal power consumption, making them capable of running for a long time on a battery [1] or on other available energy sources such as mechanical, thermal, or biochemical energy sources in situations where battery replacement is impossible [2]. This level of efficiency has made IoT devices practical for a wide range of applications in various sectors.

However, this growth has also introduced a wide variety of new security challenges with dire consequences. The mitigation of these security challenges in IoT devices has proven

to be especially difficult due to their physical and software limitations such as limited processing power, memory, and storage [3]. The growing number of IoT attacks is not only a reflection of the current growth and risk to IoT devices, but also a projection of how the future will be. Therefore, long-term protection mechanisms on IoT devices are not only necessary but also instrumental to the sustainability of IoT devices [4].

The Intrusion Detection Systems (IDS) have long been used as a means to protect devices against intrusion attacks. Security mechanisms that protect the network are implemented on the IDS and the entire network is protected. However, IDS is limited in effective installation locations due to its need to collect packets that flow over the network to detect attacks. The placement of an IDS must ensure that it can detect the traffic that flows between potential attack vectors and their targets [5].

In addition, with the rise of internet connected devices, we have also seen the rise of devices that work outside the IDS or what we would call uncontrolled environments such as public spaces, remote locations or areas with limited physical security. This means that these devices are outside the protection of the IDS, and therefore not protected against attacks. IoT devices fit into this category, while some operate inside a network where they are protected by the IDS, quite a number of IoT devices operate in open environments where they are susceptible to a wide variety of attacks such as Distributed Denial of Service (DDoS), IoT Malware, phishing, eavesdropping, and zero-day attacks [6]. Developing a Machine Learning (ML) model to detect each of the attacks would not be a viable solution. There also exists the problem of limited data on several IoT attacks such as zero-day attacks. Training a model from scratch with limited data could lead to poor performance and overfitting.

One way to solve these issues is to use transfer learning,

a mechanism in which a model trained for a certain task can be used as a base point for another model [7]. Transfer learning helps to reduce the training time, improve the model's accuracy, and allow the model to be trained on a small dataset.

Current research on the implementation of transfer learning on IoT devices relies heavily on readily available pre-trained models such as ResNet, GoogleNet, and VGG [8]. Although these models offer advantages in terms of reduced training time and improved accuracy, they also offer significant disadvantages. The pre-trained models are large in terms of tens of hundreds of megabytes, and have millions of parameters which require high computational power that is not available in the Microcontrollers [21]. Even with optimization techniques, such pre-trained models are too complex leading to large compressed model sizes, longer inference times [27] and degraded performance on the microcontroller. There is, therefore, a need to come up with smaller and efficient transfer learning models customized for the resource constrained environment that allows the Microcontrollers to benefit from the robustness of a pre-trained model.

In this study, we propose such a transfer learning model. The TL TinyML-based IDS model is developed from scratch using TinyML techniques and is therefore designed with the constraints of IoT devices in mind. The model is based on feature selection, resampling, transfer learning, and model optimization techniques to detect attacks on IoT devices. The model will later implemented on a microcontroller.

The proposed TL TinyML-based IDS model is based on a previous DDoS detection model [9] that was initially developed for the detection of DDoS attacks on IoT devices. The proposed model performed well in the detection of the attacks and was successfully implemented on an Arduino Nano 33 BLE Sense, which has only 256kb of RAM. Therefore, we sought to leverage the model's knowledge to detect other types of IoT attacks.

II. BACKGROUND

A. IoT Attacks; IoT Malware, Botnets and Zero-day Attacks

For the purpose of this study, we decided to work with datasets that contained the above named IoT attacks only.

IoT malware attacks are attacks from malware designed specifically to attack IoT devices [10]. Such attacks include Mirai, a malware that targets credentials in a device and helped launch one of the largest DDoS attack in history. According to [11] the number of IoT malware attacks in 2023 increased by 400% compared to the previous year. The manufacturing industry had the majority number of attacks accounting 54.5% of all attacks, while IoT malware in the education sector increased by almost 1000%.

To launch an IoT malware attack, an attacker creates a botnet by finding un-patched and security-vulnerable devices and installing a malware that enables him to control these devices remotely from a command and control server. The malware infected device is a bot and a botnet is basically the collection of all malware infected devices. These bots can now self-propagate, meaning that a bot can now recruit

another security-vulnerable device into the botnet without the attackers intervention [12]. When the attacker has enough bots, he instructs them to send an overwhelming floods of requests to a target, depleting its resources. This is basically a DDoS attack [13].

Zero-day attacks are attacks that aim to exploit unknown vulnerabilities before they are patched [14]. These kinds of attacks are difficult to patch as they are unknown to the developers and at the time of their occurrence, there exist no patches or defense against them. Zero-day attacks can include backdoor, worms, fuzzers, and shellcode. Early detection of zero-day attacks can minimize their impacts and decrease any losses that might be incurred.

B. IoT Devices

IoT Devices are physical devices that connect with other devices or systems over a network to exchange data. These devices range from simple home devices like a fridge, to sophisticated industrial devices such as industrial robots. Although tiny, these devices are transforming various sectors by helping automate services, improving efficiency, and saving time and cost [15].

However, some IoT devices work in open environments where they are prone to a variety of attacks such as the creation of botnets and physical attacks. Robust security measures are required to protect them from potential threats and ensure their continuous operation.

C. Machine Learning on Edge Devices

Since resource constrained devices lack the capability to handle traditional protection mechanism, they require efficient protection mechanisms. Machine Learning for Microcontrollers or TinyML is a field of ML that enables us to integrate ML with embedded devices. Implementation of TinyML requires that the ML models be optimized for a resource-constrained environment. Such optimizations may affect the model's performance especially if the model is too complex.

In this study, we used Lite Runtime (LiteRT), a platform that enable ML models to be deployed on resource-constrained devices such as IoT devices [16].

D. Transfer Learning

Transfer Learning (TL) is a method in which a model's knowledge acquired from a previous task can be reused in a new task [7]. TL can be implemented in different ways. These includes retraining the entire model on a new dataset, freezing some of the layers of the pre-trained model and training the rest on a new dataset, or freezing all the layers of the pre-trained model, adding new layers to the model and training them on a new dataset.

E. Functional API vs Sequential API

Functional API is an API that offers flexibility and control in our models by allowing us to experiment with a variety of architectures. While Sequential API creates a linear stack of layers that have exactly one input and one output per

layer, Functional API allows us to create complex models that support shared layers and multiple input and output objects. This type of architecture is essential for transfer learning since we may need to retrain, freeze or add more layers into the model [17].

F. Dataset

A total of three datasets were used to train and test the base model and the TL TinyML-based IDS model. The TON_IoT Dataset, which is a new generation of IoT and Industrial IoT (IIoT) datasets [18] was used to train the base model. The dataset contains a variety of attacks including ransomware, DoS, DDoS, Backdoors, injection, and scanning attacks.

The CICIDS2017 dataset [19] was used for training the TL TinyML-based IDS model. The dataset contains benign and attack traffic. It includes a variety of attacks such as botnet, Dos, DDoS and web attacks. While this dataset is not specifically meant for zero-day attacks, it contains a variety of attacks that make it suitable for studying patterns that could be indicative of zero-day attacks.

The MQTTset Dataset [20] was used to train and test the TL TinyML-based IDS model. The dataset relates to the MQTT protocol, which is a messaging protocol used in the IoT. The dataset contains a variety of attacks including slowITe, bruteforce, malformed data, flooding and DoS attacks. The CICIDS2017 and MQTTset datasets were used to test the TL TinyML-based IDS model to gauge its performance when faced with different types of attacks.

III. RELATED WORK

Several studies have been conducted in reference to the use of transfer learning in detection of attacks on IoT devices. A novel ensemble model was proposed by [21]. The model was a stacked Ensemble comprising of an autoencoder, Gated Recurrent Unit (GRU) and a Multi-Layer Perceptron (MLP) for malware classification. They carried out two experiments based on two different training and testing data ratios and had a testing accuracy of 99.30% and 99.43% respectively.

The authors in [22] proposed a novel approach to detect IoT malware using transfer learning. They used two pre-trained models, ResNet101 and VGG16 for feature extraction. Each pre-trained model produced a feature map. The two feature maps were then fused together and fed to a fully connected classifier to perform a classification of the malware. The proposed approach achieved an accuracy of 93.58%.

Another ResNet-based model is proposed by [23] which they refer to as P-ResNet. The P-ResNet was compared with LSTM, NN, CNN and RNN models. The proposed P-ResNet model achieved an optimal performance of 87% accuracy and performed relatively better in the other measurement indicators used compared to the other models. The performance of this model was much lower than the performance achieved by the authors in [22] when they combined ResNet101 and VGG16.

The authors in [24] proposed a deep-TL-based intrusion detection framework tailored to 5G IoT scenarios to detect known and zero-day attacks. The proposed model achieved an

overall accuracy of 97.89% and FPR of 0.05%. The proposed framework was further compared with a CNN-based IDS and the results showed that the proposed TL-based IDS improved the detection rate of zero-day attacks detection by 33.28% and known attacks by 7.1%.

The above proposed model was an improvement from the transfer learning-based IDS model proposed by [25] which was also meant to detect zero-day attacks. The TL-based model was developed to detect unlabelled R2L samples of zero-day attacks and normal traffic from the NSL-KDD dataset using labelled DoS attacks and normal traffic samples of the same dataset. The proposed method achieved an accuracy of 89.79% and FPR of 0.15% which was also an improvement from the previous studies.

An efficient-lightweight ensemble transfer learning (ELETTL-IDS) model is proposed at [26] to detect attacks in Cloud IoT environments. It is an ensemble model combining three pre-trained models, InceptionV3, MobileNetV3Small and EfficientNetV2B0. The model was able to attain the highest accuracy of 100%.

A intrusion detection bagging ensemble model consisting of three pre-trained models, Generic CNN-Q, InceptionV3-Q, MobileNet-Q is proposed by [27]. The authors applied quantization to their models, however, as the results show, the model sizes of the quantized models were too large in terms of MBs to be implemented on a microcontroller which has just a few KBs of memory and their inference times too, were high.

IV. THE PROPOSED TRANSFER LEARNING TINYML-BASED INTRUSION DETECTION FRAMEWORK

In this section we present our proposed TL TinyML-based IDS model with feature selection as summarized in Figure 1. This proposal was based on the proposal in [9], but since a simple application of transfer learning in a microcontroller is not possible, we made a few changes. This section was implemented in several phases.

A. Retraining the Base Model

The base model which had been designed to detect DDoS attacks [9], had originally been developed using a Sequential API. Unfortunately, the Sequential API model does not support multiple inputs/outputs or layer sharing, which is necessary for transfer learning [17]. We therefore had to convert the model into a Functional API model.

B. Dataset Pre-processing

Dataset pre-processing is a crucial step in any model training since it helps to make sure that the dataset is clean, consistent, and suitable for training a model. The base model had been trained to detect DDoS attacks using the TON_IoT Dataset. The same dataset was used to retrain the newly redeveloped base model.

To gauge the performance of the proposed TL TinyML-based IDS model, we used two different datasets with different kinds of attacks, the CICIDS2017 IoT Malware dataset and the

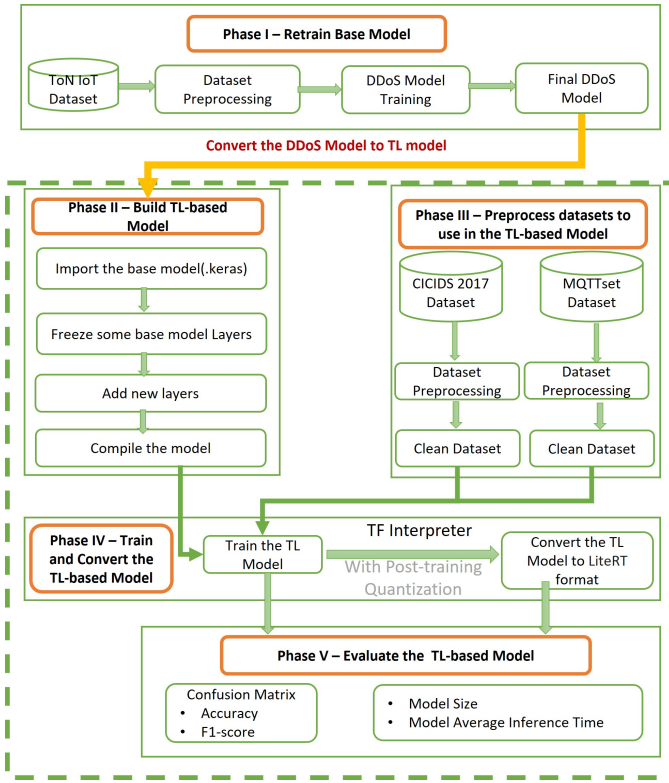


Fig. 1: Proposed Framework

MQTTset dataset. The datasets were pre-processed differently since they had different needs.

The MQTTset was pre-processed by label encoding all the categorical columns, scaling the features using the StandardScaler and dropping any duplicates. The CICIDS2017 was preprocessed by concatenating the csvs, dropping the duplicates and filling in null values with mean of the columns. The data was further scaled with the StandardScaler. The CICIDS2017 dataset was very large and had more attack traffic than normal traffic. Training a model with an unbalanced dataset could lead to a biased model. We also wanted to train the model with datasets that were within the same range of values. Therefore, using a resampling technique we sampled the normal and malware samples to ensure that we had a good distribution of the malware samples. We also reduced the dataset to fit the size of the MQTTset dataset.

C. Feature Selection

The base model had been trained using 26 features and hence we decided to use only 26 features from either of the datasets. To select just the right features from the dataset, we sought to use a feature selection method, the Analysis of Variant (ANOVA).

The use of ANOVA F-test helped improve the model's performance and reduce overfitting. Overfitting is a situation where your model learns the training data all too well, becoming unable to generalize new, unseen data well [28]. With ANOVA, whenever the model encounters a dataset with more

than 26 features, the model will automatically select only the best 26 features.

D. The TL TinyML-based IDS Model

a) *Model Development:* After we converted the base model from a sequential API into a functional API model, we saved it as a .keras model. To develop the TL TinyML-based IDS model we first imported the .keras model, froze some of its layers and added a few new layers. The frozen layers were meant to retain the knowledge from the base model while the new layers were meant to learn the new features from the new datasets. Three dense layers each with ReLU as the activation function and an output dense layer with Sigmoid as the activation function, were added. The model was compiled using Binary-Crossentropy as the loss function and Adam optimizer as the optimizer. We used a low learning rate by defining an exponential decay learning rate schedule with an initial value of 1e-3, on 64 batch size for 100 epochs with early stopping. The exponential decay learning rate allowed the model adapt the learning rate dynamically during the training process, enabling the TL TinyML-based IDS model to learn the new features, leading to better convergence.

b) *Model Training:* To test the viability of the proposed TL TinyML-based IDS framework, we trained it on a machine with enough resources using each of the pre-processed datasets separately. We wanted to find out how the proposed TL TinyML-based IDS framework would perform when presented with datasets that contain different types of settings, types of attacks and dataset sizes. Each of the dataset was split into training, testing, and inferencing sets. The training set was further split into training and validation set. Each dataset was used separately to train and test the model and each time, the model was evaluated. Cross-validation was used during training to help in assessing the model's performance and prevent overfitting.

Cross-validation is a technique that helps us evaluate the performance of a model, especially on unseen data [29]. We used the Stratified K-Fold cross-validation because it ensures that the class distribution is maintained across all the folds. It works by splitting the dataset into k equal folds and training the model on k-1 folds while testing it with the rest of the folds. We used k=5 which means that the process was repeated 5 times.

E. Converting the Model to Lite Runtime

Since our aim was to develop a lightweight transfer learning model that could run on a microcontroller, we needed to determine the model's performance when converted to a format compatible with a microcontroller. Therefore, using the TensorFlow Converter, we converted the TL TinyML-based IDS model into a Flat-file buffer format. During conversion, the model was optimized using quantization [30].

Optimization of the model is important because it reduces the size and latency which is especially important when deploying the model on an embedded device. Quantization helped us reduce the size of the model and improve the

model's inference speed by converting the model weights from floating point values in to integer values [31].

V. EXPERIMENTAL RESULTS AND DISCUSSION

In this section, we present and discuss the experimental results of our proposed TL TinyML-based framework. We used the confusion matrix, the model sizes and the average inference time of the model on each dataset to determine the performance of the model.

A. Results

A variety of methods can be used to determine the performance of a model. In our case we chose to use the Confusion Matrix which is a table that enables us to visualize how the model performed in relation to the actual expected outcome. Using the Confusion Matrix we can calculate the accuracy, precision, recall and F1-score.

The above evaluation metrics can be calculated as follows;

$$Accuracy = \frac{(TP+TN)}{(TP+TN+FP+FN)} \quad (1)$$

$$Recall = \frac{TP}{(TP+FN)} \quad (2)$$

$$Precision = \frac{TP}{(TP+FP)} \quad (3)$$

$$F1 - score = 2 * \frac{(Precision * Recall)}{(Precision + Recall)} \quad (4)$$

TABLE. I, shows the results of the TL TinyML-based model while in two different formats, the TensorFlow model and the Lite Runtime model. The displayed results are based on two different datasets.

TABLE I: Results of the TL TinyML-based IDS in detecting IoT attacks

Dataset	CICIDS2017		MQTTset	
Model Type	Accuracy	F1-Score	Accuracy	F1-Score
TFCore	97%	97%	85%	87%
LiteRT	91%	91%	85%	87%

B. Discussion

1) *Model Performance:* As shown in TABLE. I, the TL TinyML-based model performed well when trained on the two different datasets with different types of attacks. When the TL TinyML-based model was trained and tested with the CICIDS2017 dataset, it showed a higher accuracy on the TFCore model version, probably because the dataset is very similar to the dataset that was used to train the base model. However, when the model was converted to a LiteRT model, the accuracy dropped. The model, however, had a high F1-Score indicating that it was able to correctly predict both normal and attack classes.

However, when the TL TinyML-based model was trained and tested using the MQTTset, the model's performance was lower. Both the TFCore and the LiteRT had an accuracy of 85% and F1-score of 87%. Although the model did not perform as well as when trained with the CICIDS2017 dataset, probably because of the differences in the features contained in both datasets in relation to the base model, the model performed fairly well and when converted to a LiteRT version, the performance did not drop.

2) *Model Sizes:* We further used model sizes as a way to determine the model's effectiveness. Although we would like the microcontroller to be able to protect itself from attacks, we would not want the proposed framework to take so much resources that the microcontroller is unable to perform the tasks for which it was originally designed to do. Therefore, we aimed to keep the model sizes as small as possible, in just a few KBs. As shown in Figure 2, the results of training the framework with either of the datasets resulted in models of the same size in both the TFCore and LiteRT versions. The model sizes did not change despite the model being trained using different datasets. This meant that no matter which IoT attack we wanted to detect in the future, we can count on the model to produce a model size that is small enough to run on a microcontroller.

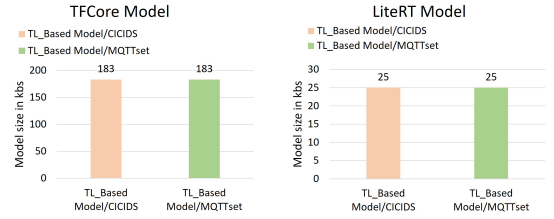


Fig. 2: Model Sizes Comparison

3) *Models Average Inference Time:* Models to be implemented on IoT devices should have as little latency as possible. Therefore, we aimed to determine the average time it would take the model to inference using either of the datasets in either the TFCore or LiteRT version. The idea was to determine whether the model would take longer when presented with datasets with different types of attack. As shown in Figure 3, the model's average inference times were relatively the same across the two different datasets. As expected, the model was much faster in the TFCore version than in the converted LiteRT version. However, the average inference time shows that no matter what attack the model would try to detect, the inference time would be low.

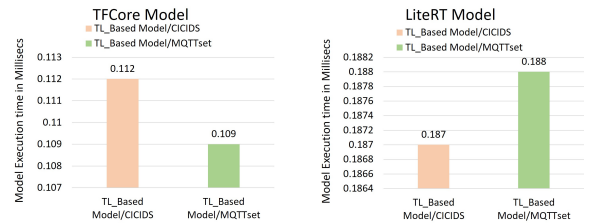


Fig. 3: Models Average Inference Time

C. Comparison with Existing Frameworks

We further compared the performance of our proposed TL TinyML-based IDS model with the other existing models discussed in the related work section of this proposal. The researchers created TL models for IoT devices, however, they did not focus on a real world implementation of their proposed

TL models. As noted in TABLE. II, some researchers did not consider aspects such as the model size, model optimization or even the inference time. The authors in [27] considered the size of their model and even quantized their model, however, their proposal resulted in high inference times and large quantized model sizes that were in MBs compared to a few KBs of memory on a microcontroller.

Our proposed framework, on the other hand, has a small quantized model size and low average inference time, and therefore can be implemented on a microcontroller. TABLE. II shows the best results of the proposals from the related work section. "Yes" means that the proposal implemented the feature and "No" means it did not.

TABLE II: Comparison of the TL TinyML-based IDS to other related works

Work	Feature Selection	Model Size	Quantization	Inference Time	Accuracy
CNN-Model [21]	No	No	No	No	99%
ResNet101 & VGG16 [22]	No	No	No	Yes	93.58%
P-ResNet [23]	No	No	No	Yes	87%
TL-Based Prototype [25]	No	No	No	No	89.79%
ELETL-IDS [26]	Yes	No	No	Yes	100%
Bagging Ensemble [27]	No	Yes	Yes	Yes	99.9%
TL TinyML-based IDS	Yes	Yes	Yes	Yes	97%

VI. CONCLUSION

In this study, we proposed a TL TinyML-based intrusion detection framework with feature selection. We aimed to create an efficient, lightweight transfer learning framework, customized for the IoT environment. The framework was tested by training it on two different datasets with different kinds of attacks. From the results, the framework was able to detect the different kinds of IoT attacks with good accuracy, small model sizes and low average inference times in its original format and in its quantized format. In the future, we will implement the TL TinyML-based framework on an Arduino Nano 33 BLE Sense for testing.

REFERENCES

- [1] K. Hasan, N. Tom, M.R. Yuce, "Navigating Battery Choices in IoT: An Extensive Survey of Technologies and Their Applications", *MDPI Batteries*, 9, 580, 2023.
- [2] A. Somov and R. Giaffreda, *Powering IoT Devices: Technologies and Opportunities*, Available: <https://iot.ieee.org/articles-publications...917>.
- [3] A. Haroon, "Constraints in the IoT: the world in 2020 and beyond", *International Journal of Advanced Comp. Science & Applications*, Vol.7, No.11, 2016.
- [4] G. Shirvani and S. Ghasemshirazi, "Towards Sustainable IoT: Challenges, Solutions, and Future Directions for Device Longevity", *arXiv preprint arXiv:2405.16421*, 2024.
- [5] R. Puzis, M. Tubi, Y. Elovici, C. Glezer, and S. Dolev, "A decision support system for placement of intrusion detection and prevention devices in large-scale networks", *ACM Transactions on Modeling & Comp. Simulation*, Vol.22, No. 1, pp. 1-26, 2011.
- [6] A. Tabassum and W. Lebdia, "Security Framework for IOT devices against Cyber-attacks", *arXiv preprint arXiv:1912.01712*, 2019.
- [7] F. Zhuang et al., "A Comprehensive Survey on Transfer Learning", *Proceedings of the IEEE*, Vol. 109, No. 1, pp. 43-76, 2021.
- [8] V. Lendave, *Top 4 Popular Transfer Learning Models – Detailed Comparison*, Available: <https://analyticsindiamag.com/ai-mysteries/a-comparison-of-4-popular-transfer-learning-models/>
- [9] J. Mwaura, A. Shunsuke and K. Kakizaki, "A Study on DDoS Attacks Detection on IoT Devices Using Machine Learning for Microcontrollers", *IEEE Inter. Conf. on Consumer Electronics*, pp. 1-4, 2024.
- [10] J. S. Perry, *Anatomy of an IoT malware attack*, Available: <https://developer.ibm.com/articles/iot-anatomy-iot-malware-attack/>
- [11] Zscaler Zero Trust Exchange™ platform, *Zscaler ThreatLabz 2023 ... Increase in IoT Malware Attacks*, Available: www.zscaler.com/
- [12] N. Hoque, D. Bhattacharyya and J. Kalita, "Botnet in DDoS Attacks: Trends and Challenges", *IEEE Communications Surveys & Tutorials*, Vol. 17, Issue 4, pp. 2242-2270, 2015.
- [13] Microsoft Team, *What is a DDoS attack?*, Available: www.microsoft.com/.../what-is-a-ddos-attack.
- [14] K. Vaisla, "Analyzing of Zero Day Attack and its Identification Techniques", *Proceedings of First Inter. Conf. on Advances in Comp. & Comm. Eng.* 1. pp. 11-13, 2014.
- [15] A. Chowdhury, S. A Rauta, "Benefits, challenges, and opportunities in adoption of Industrial IoT", *Inter. Journal of Computational Intelligence & IoT*, Vol. 2, No. 4, 2019.
- [16] TensorFlow Team, "Deploy machine learning models on mobile and edge devices", Available: <https://www.tensorflow.org/lite>.
- [17] Keras Team, "Models API", Available: <https://keras.io/api/models/>.
- [18] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood and A. Anwar, "TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Intrusion Detection Systems", *IEEE Access*, Vol. 8, pp. 165130-165150, 2020.
- [19] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization", *4th Inter. Conf. on Information Systems Security and Privacy (ICISSP)*, Portugal, January 2018.
- [20] I. Vaccari, G. Chiola, M. Aiello, M. Mongelli and E. Cambiaso, "MQTTset, a New Dataset for Machine Learning Techniques on MQTT", *Sensors* Vol.20, No. 22: 6578, 2020.
- [21] P. Panda, O. K. C U, S. Marappan, S. Ma, M. S, and D. Veasani Nandi, "Transfer Learning for Image-Based Malware Detection for IoT", *Sensors* Vol. 23, No. 6: 3253, 2023.
- [22] Q. Kh Kadhim et al., "IOT-MDEDTL: IoT Malware Detection based on Ensemble Deep Transfer Learning", *Majlesi Journal of Electrical Engineering*, Vol. 16, No. 3, pp. 47-54, 2022.
- [23] S. T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed and R. Islam, "Dependable Intrusion Detection System for IoT: A Deep Transfer Learning Based Approach", *IEEE Transactions on Industrial Informatics*, vol. 19, No. 1, pp. 1006-1017, 2023.
- [24] E. Rodríguez et al., "Transfer-Learning-Based Intrusion Detection Framework in IoT Networks", *Sensors* Vol. 22, No. 15: 5621, 2022.
- [25] N. Sameera, M. Shashi, "Transfer Learning Based Prototype for Zero-Day Attack Detection", *Int. Journal of Eng. and Advanced Technology*, Vol. 8, Issue. 4, pp. 1326-1328, 2019.
- [26] O. D. Okey, D. C. Melgarejo, M. Saadi, R. L. Rosa, J. H. Kleinschmidt and D. Z. Rodríguez, "Transfer Learning Approach to IDS on Cloud IoT Devices Using Optimized CNN", *IEEE Access*, vol. 11, pp. 1023-1038, 2023.
- [27] B. Ahmad, Z. Wu, Y. Huang, and S. U. Rehman, "Enhancing the security in IoT and IIoT networks: An intrusion detection scheme leveraging deep transfer learning", *Knowledge-Based Systems*, vol. 305, p. 112614, Dec. 2024.
- [28] X. Ying, "An overview of overfitting and its solutions", *Journal of physics: Conference series*, Vol. 1168, pp.022022, 2019.
- [29] D. Anguita, L. Ghelardoni, A. Ghio, L. Oneto and S. Ridella, "The 'K' in K-fold Cross Validation", *European Symposium on Artificial NN, Comp. Intelligence & ML*, pp. 441-446, 2012.
- [30] TensorFlow Team, *Model conversion*, Available: <https://www.tensorflow.org/lite/models/convert>.
- [31] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A White Paper on Neural Network Quantization", Jun. 2021.